protoST — talk script (FAST, 15/10/2026)

Length: a 30-minute core; optional modules up to 60. The performance figures come from these reports, all of synthetic benchmarks on one laptop (Ryzen 5 5500U):

- `benchmarks/reports/2026-10-12-protocore-2.14.6.md` (12/10): the published protoST binary on protoCore 2.14.6 against the same binary on 2.14.5, and CPython 3.14 on the same machine. Startup, the CPython comparisons, the actors, demo 2 and K5's memory come from it;
- `benchmarks/reports/2026-10-12-adaptive-heap-default.md` (12/10): 0.6.2 against 0.6.1, the adaptive heap controller (actors, memory, pauses);
- `benchmarks/reports/2026-10-11-actor-regression-bisection.md` (11/10): the measurement noise.

If a figure is not in one of those reports, it is not said.

Before starting: terminal with a large font, `cd docs/talks/2026-10-15-fas/demos` in the repository (or, with the package installed, `cd /usr/share/protoST/talks/2026-10-15-fas/demos`); `protost --version` must print `protoST 0.6.5`; demo 4 needs `python3`. The fallback recordings are in the repository's `../recordings/` (see the checklist).

Core (30 minutes)

0:00 — Who and why (2 min)

- This is an experiment looking for enthusiasts, not a product looking for users.
- protoST is not a better Smalltalk and does not compete with the live environment. It is something else: being able to read and write the Smalltalk you know in places Smalltalk does not reach today.
- It is built by one person using AI agents as tools; everything I show is in the repository and can be verified.

0:02 — Questions, not answers (3 min)

Put them to the audience as questions (do not assert them on their behalf):

- What does it take you today to put an image behind something else as a service?
- How many cores does your Smalltalk code use when the machine has twelve?
- How do you integrate what you write in Smalltalk with the rest of the stack (Python, JavaScript, queues, databases)?
- How do you share code as plain text with someone who does not use an image?

We do not say that protoST solves all of that. We say that it opens a different way to try.

0:05 — Why it is possible now: protoCore (5 min)

- protoCore: an object core in C++ on which several languages run (JavaScript, Python, Clojure, Scala, protoST).
- Three properties, each with a simple example:
  - immutable structures with structural sharing: a "modified" list is a new version that shares almost everything with the previous one;
  - native threads with no global lock, and a concurrent GC;
  - a mutable object is an atomic reference to an immutable snapshot: every read sees a consistent snapshot.
- From Smalltalk, objects still change: `at:put:` on an `Array` or `add:` on an `OrderedCollection` modify the object by installing a new snapshot. Strings do not: a String is a protoCore `ProtoString` (a rope, a value with no in-place modification).
- Practical consequence: passing an object to another thread means passing a reference, without copying it. The object is shared: what one side changes, the other sees on its next read.

0:10 — DEMO 1: the code you already know (5 min)

```
./run_demo.sh 01-familiar-code.st
```

Show the file before running it (classes, `printOn:`, blocks, non-local return in `Bank>>find:`, a custom exception with `retry`, `ZeroDivide` with `resume: 0`, exact fractions, `LargeInteger`). Points to highlight:

- it is a text file; a blank line ends a method;
- `new` sends `initialize`, `printOn:` governs how everything is printed;
- the last line prints the total (270): a script shows only what it prints.

Fallback: `scriptreplay` of `recordings/01-familiar-code`.

0:15 — Actors and real parallelism (3 min)

- Any object becomes an actor with `asActor`; every message returns a `Future` at once (except those that ask about the reference itself: identity, equality, `hash`, `isNil`, `isActor`, `printString`, which the proxy answers immediately); the actor processes one message at a time, so its state is plain instance variables, with no locks.
- Actors share a pool of native threads.
- `wait`, `thenDo:`, `whenAll:`. An error inside the actor comes back, with its class, to whoever is waiting.

0:18 — DEMO 2: the same computation on one thread and on twelve actors (3 min)

```
./run_demo.sh 02-actors-parallelism.st
```

Both results agree (4946 primes) and the times measured at that moment are shown; the last line (`time speedup: ...x`) is the speedup. State the one that appears on screen, not one from memory: it depends on the machine and its load. On the talk machine, idle and in AC mode, with protoCore 2.14.6: a median of 2.4× (2.1–2.6×), with one thread at about 715 ms and the twelve actors at about 295 ms. Point out: the code of the computation did not change; what changed is who runs it. If someone asks why it is less than the 2.8× of before: protoCore 2.14.6 made the single-thread code faster (see hard-questions.md).

Fallback: `recordings/02-actors-parallelism`.

0:21 — Worlds it could open up, and DEMO 4 (6 min)

Ideas, not promises:

- **Digital twins**: one actor per physical component; each event is processed atomically; the components talk through messages. (If there is time, DEMO 3.)
- **Services and connections to other systems**: a process that starts in tens of milliseconds, reads files and standard input, speaks HTTP (client and server, https too) and TCP/UDP, and runs other programs; no image to deploy. The names are Pharo's: `'data.csv' asFileReference contents`, `Stdio stdin nextLine`, `Smalltalk arguments`.
- **Living alongside other runtimes**: since protoCore 2.6, a runtime embedded in a process can read and write another runtime's objects without copying them. What is tested is one direction: protoScala reads and writes protoST objects; the reverse (protoST using another runtime's objects or modules) does not work yet (`docs/INTEROP.md` §0). The installed binaries do not load each other yet. Say it exactly like that.

DEMO 4: `./run_demo.sh 04-connected-twin.st`

A Python program emits sensor readings, one JSON object per line; the protoST twin reads them on standard input (it is a Unix filter: `python3 04-connected-twin.feed | protost 04-connected-twin.st`), the pump is an actor, its state is served over HTTP while it runs, and every reading is logged to a file. Show the code: `Stdio stdin linesDo:`, `JSON parse:`, `HTTPServer on:handler:`, `writeStreamDo:`. Point out: another language delivers the data, protoST processes it with actors and exposes it over HTTP; none of it needed an image.

Fallback: `recordings/04-connected-twin`.

0:27 — What protoST is not (2 min)

Read the headlines of chapter 14 of the tutorial: no image or browser, a blank line ends a method, immutable strings, short symbols equal to strings, thin metaclass, `thisContext` not supported, an actor that is waiting does not handle other messages (and a cycle of waits is reported as an error). Show that it is all written down and verified.

0:29 — How to take part (1 min)

- Repository, tutorial (chapter 14 is for you), `docs/STATUS.md` with what is open.
- What helps most: real programs written the way you would write them, and reports of everything that surprises you.
- Questions.

Modules (up to 60 minutes)

M1 — Inside protoCore (10 min)

64-byte cells, integers inline in the pointer, immutable AVL lists and ropes, `ProtoThread` and how protoST uses it for the workers. The atomic snapshot of a mutable object (drawn on the whiteboard).

The collector as it stands in protoCore 2.14.6: concurrent; the sweep walks several chains at once and can add helper threads; since 2.14.6 the mark work list has a fixed size (8 MiB). Since 0.6.2 protoST uses protoCore's adaptive heap controller (up to 0.6.1, a fixed 640 MB ceiling): faster on the actor programs, with a fifth to a half of the memory. The price: peak memory is no longer capped (a loop that keeps allocating reached 0.88–3.5 GB); `PROTOCORE_HEAP_LIMIT_CELLS` fixes it. The way there needed a fix of protoST's own: each stop-the-world pause scanned a whole 1 M-slot frame array per thread (pauses of up to 20–31 ms with six to twelve workers); since 0.6.2 the array grows on demand and pauses stay below 0.27 ms on the benchmarks. Under the fixed 640 MB ceiling, a 500,000-element array took the process to 1,165 MB of RSS because of the mark work list; protoCore 2.14.6 bounds it and the peak falls to 690 MB. Present it as a measurement (`KNOWN_ISSUES.md`, K5 and K6).

M2 — The digital twin step by step (8 min)

`./run_demo.sh 03-digital-twin.st`: the pump as a state machine in an actor, three sensors with 50 ms of simulated I/O read in parallel on every cycle (the time per cycle on screen is around 50 ms, not 150), the alarm that stops the pump when it goes above 75 degrees. Walk through the code.

Fallback: `recordings/03-digital-twin`.

M3 — Performance and method (6 min)

- Every benchmark verifies its result; the harness fails if the result is missing or does not match (a bug in another project that looked like an "improvement" because it was not verified was shown).
- Synthetic benchmarks on a laptop (Ryzen 5 5500U), report of 12/10 on protoCore 2.14.6:
  - startup 25 ms (CPython 32–35 ms);
  - against CPython, whole process with the same N: geometric mean 3.2–3.3×, dominated by startup;
  - the work alone (startup subtracted): from about 4–7× (integer loop) to 30–35× (`fib`) and 55–64× (exceptions); the geometric mean came out between 14× and 20× depending on the run, because with small N the subtraction is sensitive to noise; give it as a range;
  - twelve actors: 2.8× with the pool against one worker; `saturation_big`, 2.5× with four workers and 2.9× with six, no ceiling up to six;
  - protoCore 2.14.6 against 2.14.5: demo 2 45 % faster on one thread and 35 % faster with twelve actors; exceptions 7 % faster; the rest the same within the noise;
  - 0.6.2 enabled the adaptive heap controller (heap report of 12/10: actors 16–43 % faster, a fifth to a half of the memory, and the peak no longer capped); 0.6.1 fixed collection in recursions that never loop (fib 29 ran out of memory).
- Say where it is slower (recursive method dispatch, exceptions) without dressing it up.

M4 — Roadmap and open questions (6 min)

Runtimes loading each other (the core allows it since protoCore 2.6), `thisContext`, tooling (a DAP debugger exists), message-send performance, WebSockets and HTTP/2. Ask the audience what would be useful to them first.